

longcat

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
LongCat API Key 自动获取脚本 (Python + Selenium)
基于 DuckMail 邮箱 + Keeta 登录流程 + 验证码提取
"""

import os
import re
import time
import json
import html
import random
import urllib.request
import urllib.error
import subprocess
from datetime import datetime
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import (
    TimeoutException,
    NoSuchElementException,
    ElementClickInterceptedException,
    StaleElementReferenceException
)

# ===== 配置区域 =====
CONFIG = {
    "KEY_FILE": os.path.join(os.path.dirname(os.path.abspath(__file__)), "long_cat_key.txt"),
    "MAX_LOOPS": 500, # 循环次数
    # --- DuckMail API 配置 ---
    "DUCKMAIL_API_BASE": "https://api.duckmail.sbs",
    "DUCKMAIL_REGISTER_PATH": "/accounts",
    "DUCKMAIL_TOKEN_PATH": "/token",
    "DUCKMAIL_MESSAGES_PATH": "/messages",
    "DUCKMAIL_BEARER": os.getenv(
        "DUCKMAIL_BEARER",
        "dk_744ab50b9d77cdf83dff6a8f8097475dacddb3e5b6b36ff38eccf0355a5e1521",
    ),
    "DUCKMAIL_EMAIL_MIN_LEN": 8,
    "DUCKMAIL_EMAIL_MAX_LEN": 13,
    "DUCKMAIL_PASSWORD_LEN": 12,
    "MAIL_POLL_INTERVAL": 3,
    "MAIL_POLL_MAX": 30,
    # --- Selenium 配置 ---
    "HEADLESS": False,
    "CHROMEDRIVER_PATH": os.getenv("CHROMEDRIVER_PATH", "/opt/homebrew/bin/chromedriver"),
    "USE_PERSISTENT_PROFILE": True, # 每次循环清除数据, 使用无痕或临时配置更好
    "USER_DATA_DIR": os.getenv(
        "CHROME_USER_DATA_DIR",
        os.path.expanduser("~/Library/Application Support/Google/Chrome"),
    ),
}
```

```

),
"PROFILE_DIR": "Default",
"INCOGNITO": False,
# --- 目标 URL ---

"LOGIN_URL": "https://passport.mykeeta.com/pc/login?
locale=en&region=HK&joinkey=1101498_851697727&token_id=5oTEq210UBLUcm4tcuuy6A&service=consumer&
risk_cost_id=119801&theme=longcat&cityId=810001&backurl=https%3A%2F%2Flongcat.chat%2Fapi%2Fv1%2
Fuser-loginV3%3Furl%3Dhttps%253A%252F%252Flongcat.chat%252Fplatform%252Fprofile",
"API_KEY_URL": "https://longcat.chat/platform/api_keys",
# 常用 User-Agents 池
"USER_AGENTS": [
    "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/122.0.0.0 Safari/537.36",
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/123.0.0.0 Safari/537.36",
    "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101 Firefox/115.0",
],
}
# =====

def log(msg: str):
    """日志输出"""
    print(f"[{datetime.now().strftime('%H:%M:%S')}] {msg}")

def generate_random_prefix() → str:
    """生成 8-13 位随机字符作为邮箱前缀"""
    chars = "abcdefghijklmnopqrstuvwxyz0123456789"
    length = random.randint(
        CONFIG["DUCKMAIL_EMAIL_MIN_LEN"],
        CONFIG["DUCKMAIL_EMAIL_MAX_LEN"],
    )
    return "".join(random.choice(chars) for _ in range(length))

def generate_random_password() → str:
    """生成随机密码"""
    chars = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
    length = CONFIG["DUCKMAIL_PASSWORD_LEN"]
    return "".join(random.choice(chars) for _ in range(length))

def request_json(method: str, url: str, headers=None, payload=None, timeout=15):
    """
    发送 JSON 请求, 返回 (status_code, raw_text, json_data)
    """
    headers = headers.copy() if headers else {}
    data = None
    if payload is not None:
        data = json.dumps(payload).encode("utf-8")
        headers.setdefault("Content-Type", "application/json")

    # 添加 User-Agent 防止被某些 API 屏蔽
    headers.setdefault("User-Agent", "Mozilla/5.0 (compatible; LongCatBot/1.0)")

    req = urllib.request.Request(url, data=data, headers=headers, method=method)

    try:
        with urllib.request.urlopen(req, timeout=timeout) as resp:

```

```

        raw_text = resp.read().decode("utf-8")
        try:
            json_data = json.loads(raw_text) if raw_text else None
        except json.JSONDecodeError:
            json_data = None
        return resp.getcode(), raw_text, json_data
except urllib.error.HTTPError as e:
    raw_text = ""
    try:
        raw_text = e.read().decode("utf-8")
    except Exception:
        pass
    try:
        json_data = json.loads(raw_text) if raw_text else None
    except json.JSONDecodeError:
        json_data = None
    return e.code, raw_text, json_data
except Exception as e:
    log(f"请求异常: {e}")
    return 0, str(e), None

```

```
def create_duckmail_account() → tuple:
```

```
    """
```

```
    通过 DuckMail API 创建邮箱账号, 返回 (email, password)
```

```
    """
```

```
    log("开始创建 DuckMail 邮箱账号...")
```

```
    api_base = CONFIG["DUCKMAIL_API_BASE"].rstrip("/")
```

```
    register_url = f"{api_base}{CONFIG['DUCKMAIL_REGISTER_PATH']}"
```

```
    headers = {"Authorization": f"Bearer {CONFIG['DUCKMAIL_BEARER']}"}
```

```
    for attempt in range(5):
```

```
        email_local = generate_random_prefix()
```

```
        email = f"{email_local}@duckmail.sbs"
```

```
        password = generate_random_password()
```

```
        payload = {"address": email, "password": password}
```

```
        status, raw_text, data = request_json(
```

```
            "POST",
```

```
            register_url,
```

```
            headers=headers,
```

```
            payload=payload,
```

```
            timeout=15,
```

```
        )
```

```
        if status == 201 or (status == 200 and isinstance(data, dict) and data.get("address")
== email):
```

```
            log(f"创建邮箱成功: {email}")
```

```
            return email, password
```

```
    if isinstance(data, dict):
```

```
        error_message = data.get("message", "")
```

```
        if "Email address already exists" in error_message:
```

```
            log("邮箱已存在, 重试...")
```

```
            continue
```

```
    log(f"创建邮箱失败 (status={status}), 响应: {raw_text}")
```

```
    time.sleep(1)
```

```
    raise Exception("无法创建 DuckMail 邮箱")
```

```
def get_mail_token(email: str, password: str) → str:
    """
    获取 DuckMail token
    """
    api_base = CONFIG["DUCKMAIL_API_BASE"].rstrip("/")
    token_url = f"{api_base}{CONFIG['DUCKMAIL_TOKEN_PATH']}"
    payload = {"address": email, "password": password}

    status, raw_text, data = request_json("POST", token_url, payload=payload, timeout=15)
    if status == 200 and isinstance(data, dict) and data.get("token"):
        return data["token"]

    raise Exception(f"获取 DuckMail token 失败: {raw_text}")

def get_verification_code(mail_token: str) → str:
    """
    从 DuckMail API 获取 4 位验证码
    """
    api_base = CONFIG["DUCKMAIL_API_BASE"].rstrip("/")
    messages_url = f"{api_base}{CONFIG['DUCKMAIL_MESSAGES_PATH']}"
    headers = {"Authorization": f"Bearer {mail_token}"}

    log("等待验证码邮件...")
    for i in range(CONFIG["MAIL_POLL_MAX"]):
        time.sleep(CONFIG["MAIL_POLL_INTERVAL"])

        status, raw_text, data = request_json("GET", messages_url, headers=headers, timeout=15)
        if status != 200:
            continue

        messages = None
        if isinstance(data, dict):
            messages = data.get("hydra:member") or data.get("member") or data.get("data")
        elif isinstance(data, list):
            messages = data

        if not messages:
            continue

        # 获取最新的一封邮件
        first_msg = messages[0]
        msg_id = None
        if isinstance(first_msg, dict):
            msg_id = first_msg.get("id") or first_msg.get("@id")

        if not msg_id:
            continue

        if isinstance(msg_id, str) and msg_id.startswith("/messages/"):
            msg_id = msg_id.split("/")[-1]

        # 获取邮件详情
        detail_url = f"{messages_url}/{msg_id}"
        status, detail_raw, detail_data = request_json("GET", detail_url, headers=headers)

        if status == 200 and isinstance(detail_data, dict):
            text = detail_data.get("text") or detail_data.get("html") or ""
```

```

# 解码 HTML 实体
text = html.unescape(text)

# 使用正则匹配验证码
# 模式 1: **1234** (Keeta 常见格式)
match = re.search(r"\*\*(\d{4})\*\*", text)
if not match:
    # 模式 2: verification code: 1234
    match = re.search(r"verification code[:\s]*(\d{4})", text, re.IGNORECASE)
if not match:
    # 模式 3: 直接找 4 位数字
    match = re.search(r"\b(\d{4})\b", text)

if match:
    code = match.group(1)
    log(f"提取到验证码: {code}")
    return code

log(f"邮件已收到, 但未匹配到验证码 (第 {i+1} 次)...")

return ""

# ===== Selenium 相关函数 =====

def init_browser():
    """初始化浏览器"""
    options = Options()
    if CONFIG["HEADLESS"]:
        options.add_argument("--headless=new")

    # 禁用自动化特征
    options.add_argument("--disable-blink-features=AutomationControlled")
    options.add_argument("--no-sandbox")
    options.add_argument("--disable-infobars")

    ua = random.choice(CONFIG["USER_AGENTS"])
    options.add_argument(f"user-agent={ua}")

    service = Service(CONFIG["CHROMEDRIVER_PATH"]) if CONFIG["CHROMEDRIVER_PATH"] else None

    driver = webdriver.Chrome(service=service, options=options)
    driver.set_window_size(1280, 800)

    # 注入 JS 绕过简单的 webdriver 检测
    driver.execute_cdp_cmd("Page.addScriptToEvaluateOnNewDocument", {
        "source": """
            Object.defineProperty(navigator, 'webdriver', {get: () => undefined});
        """
    })

    return driver

def human_click(driver, element):
    """模拟人类点击"""
    try:
        ActionChains(driver).move_to_element(element).pause(random.uniform(0.1,
0.3)).click().perform()
    except Exception:

```

```

driver.execute_script("arguments[0].click();", element)

def wait_and_find(driver, by, value, timeout=10):
    return WebDriverWait(driver, timeout).until(EC.presence_of_element_located((by, value)))

def wait_and_find_clickable(driver, by, value, timeout=10):
    return WebDriverWait(driver, timeout).until(EC.element_to_be_clickable((by, value)))

def automation_loop():
    driver = None
    try:
        driver = init_browser()

    for loop_index in range(1, CONFIG["MAX_LOOPS"] + 1):
        log(f"≡ 开始第 {loop_index} 轮任务 ≡")

        # 1. 清理 Cookie 保证环境纯净
        driver.delete_all_cookies()
        try:
            driver.execute_cdp_cmd("Network.clearBrowserCache", {})
            driver.execute_cdp_cmd("Network.clearBrowserCookies", {})
        except:
            pass

        # 2. 创建邮箱
        try:
            email, password = create_duckmail_account()
            mail_token = get_mail_token(email, password)
        except Exception as e:
            log(f"邮箱准备失败: {e}")
            time.sleep(2)
            continue

        # 3. 打开登录页
        log("访问登录页...")
        driver.get(CONFIG["LOGIN_URL"])

        # 智能判断登录模式
        log("判断当前登录模式...")
        try:
            # 尝试快速查找邮箱输入框 (2秒超时)
            WebDriverWait(driver, 2).until(
                EC.presence_of_element_located((By.CSS_SELECTOR, "input[placeholder='Email address']")))
            log("直接检测到邮箱输入框, 无需切换")
        except TimeoutException:
            # 如果没找到邮箱输入框, 尝试寻找切换按钮
            log("未检测到邮箱输入框, 尝试查找 'Continue with email' 按钮...")
            try:
                email_switch_btn = WebDriverWait(driver, 3).until(
                    EC.element_to_be_clickable((By.XPATH, "//*[contains(text(), 'Continue with email') or contains(text(), 'Log in with Email')]")))
                log("找到了切换按钮, 正在点击...")
                human_click(driver, email_switch_btn)
                time.sleep(1)

```



```
except TimeoutException:
    log("未找到切换按钮，可能页面结构已变或加载过慢")
    driver.save_screenshot(f"login_switch_fail_{loop_index}.png")

# 4. 输入邮箱
log("输入邮箱...")
try:
    from selenium.webdriver.common.keys import Keys
    log("输入邮箱...")
    # 等待 Email 输入框（此时应该有了）
    email_input = wait_and_find(driver, By.CSS_SELECTOR, "input[placeholder='Email address']", timeout=10)

    # 尝试用 Ctrl+A/Cmd+A + Backspace 确保清理彻底
    try:
        email_input.click()
        time.sleep(0.2)
        email_input.send_keys(Keys.COMMAND, "a") # Mac 环境用 COMMAND
        time.sleep(0.2)
        email_input.send_keys(Keys.BACKSPACE)
    except:
        email_input.clear()

    time.sleep(0.5)
    email_input.send_keys(email)
    time.sleep(0.5)

    # 点击 Continue
    # MyKeeta 的 Continue 按钮通常是一个 div
    continue_btn = wait_and_find_clickable(driver, By.XPATH,
                                           "//*[contains(text(), 'Continue')]")
    human_click(driver, continue_btn)
except Exception as e:
    log(f"登录步骤异常：{e}")
    driver.save_screenshot(f"login_error_{loop_index}.png")
    continue

# 5. 等待验证码并输入
log("等待并获取验证码...")
code = get_verification_code(mail_token)
if not code:
    log("未获取到验证码，跳过本轮")
    continue

log(f"输入验证码：{code}")
try:
    # 等待输入框出现
    wait_and_find(driver, By.CSS_SELECTOR, "input.oversea-verification-code-input")
    inputs = driver.find_elements(By.CSS_SELECTOR, "input.oversea-verification-code-input")

    if len(inputs) < 4:
        log(f"验证码输入框数量异常：{len(inputs)}")
        # 尝试另外一种查找方式
        inputs = driver.find_elements(By.XPATH, "//*[contains(@class, 'verification-code')]")

    if len(inputs) ≥ 4:
        for i, digit in enumerate(code[:4]):
            inputs[i].send_keys(digit)
```



```
        time.sleep(random.uniform(0.05, 0.2))
    else:
        log("无法找到 4 个验证码输入框, 尝试直接输入到第一个")
        inputs[0].send_keys(code)

    # 点击验证码页面的 Continue
    time.sleep(1)
    log("点击验证码确认按钮...")
    # 重新查找 Continue, 因为页面可能刷新了
    continue_btn_2 = wait_and_find_clickable(driver, By.XPATH,
        "//div[contains(text(), 'Continue')]")
    human_click(driver, continue_btn_2)

except Exception as e:
    log(f"验证码输入异常: {e}")
    continue

# 6. 跳转到 API Key 页面
log("等待登录跳转...")
time.sleep(5) # 等待登录完成重定向

log("前往 API Key 管理页...")
driver.get(CONFIG["API_KEY_URL"])

# 7. 创建 API Key
log("创建 API Key...")
try:
    # 先等待整个区域加载完成
    wait_and_find(driver, By.CSS_SELECTOR, ".apikeys, .box", timeout=10)
    time.sleep(1)

    # 尝试多种选择器查找创建按钮 (Meituan Design / MTD)
    create_btn = None
    selectors = [
        "//button[@type='button' and contains(., '创建API Key')]",
        "//button[contains(@class, 'mtd-btn') and contains(., '创建API Key')]",
        "//button[contains(., '创建API Key')]",
    ]

    # 轮询查找, 总共等待 10 秒
    end_time = time.time() + 10
    while time.time() < end_time:
        for xpath in selectors:
            try:
                btn = driver.find_element(By.XPATH, xpath)
                if btn.is_displayed():
                    create_btn = btn
                    log(f"找到创建按钮, 使用选择器: {xpath}")
                    human_click(driver, create_btn)
                    break
            except:
                pass
        if create_btn:
            break
        time.sleep(1)

    if not create_btn:
        raise Exception("无法找到创建按钮")

    # 弹窗输入 Name
```



```

time.sleep(1)
# mtd-input

name_input = wait_and_find(driver, By.CSS_SELECTOR,
"input[placeholder*='Name'], input[placeholder*='名称'], .mtd-input", timeout=5)
name_input.send_keys("test")

# 点击确认
confirm_selectors = [
    "//div[contains(@class, 'mtd-modal')]/button[contains(., '确定')]",
    "//div[contains(@class, 'mtd-modal')]/button[contains(., '确定')]",
    "//div[contains(@class, 'mtd-modal')]/button[contains(., '创建')]",
    "//div[contains(@class, 'mtd-modal')]/button[contains(., 'Create')]",
    "//button[contains(., '确定')]",
    "//button[contains(., '确定')]",
]

confirm_btn = None
for xpath in confirm_selectors:
    try:
        confirm_btn = driver.find_element(By.XPATH, xpath)
        if confirm_btn.is_displayed():
            log(f"找到确认按钮: {xpath}")
            human_click(driver, confirm_btn)
            break
    except:
        continue
if not confirm_btn:
    # 尝试找最后一个 primary button 在 modal 里
    try:
        btns = driver.find_elements(By.CSS_SELECTOR, ".mtd-modal .mtd-btn-
primary")

        if btns:
            human_click(driver, btns[-1])
            log("尝试点击 Modal 中的最后一个 Primary 按钮")
    except:
        pass

except Exception as e:
    log(f"创建 API Key 步骤异常: {e}")
    driver.save_screenshot(f"create_key_fail_{loop_index}.png")
    with open(f"debug_page_source_{loop_index}.html", "w") as f:
        f.write(driver.page_source)

# 8. 提取并保存 Key
log("提取 API Key...")
time.sleep(2)
try:
    # 刷新一下页面确保 Key 显示
    driver.refresh()
    time.sleep(3)

    # 尝试去点击 "显示/隐藏" 按钮 (mtdicon-visibility-on-o / off-o)
    # 页面源码显示: <i data-v-5240be39="" class="icon mtdicon mtdicon-visibility-on-
o"></i>

    try:
        visibility_btn = driver.find_element(By.CSS_SELECTOR, ".mtdicon-visibility-
on-o, .mtdicon-visibility-off-o")
        if visibility_btn.is_displayed():
            log("找到显示 Key 的按钮, 点击以取消掩码...")
            human_click(driver, visibility_btn)

```

```

        time.sleep(1)
    except:
        log("未找到显示 Key 的按钮，可能已经是明文或无此按钮")

    # 获取页面原文找 ak_
    page_source = driver.page_source
    # 掩码后的 Key 是 ak_*****, 我们需要找完整的
    # 如果点击了显示按钮，应该能看到完整 Key
    found_keys = re.findall(r'(ak_[a-zA-Z0-9_\-]+)', page_source)

    key_to_save = None
    if found_keys:
        # 过滤掉掩码的 (包含 *)
        valid_keys = [k for k in found_keys if '*' not in k and len(k) > 20]
        if valid_keys:
            key_to_save = valid_keys[0]
            log(f"正则提取到 Key: {key_to_save}")

    if not key_to_save:
        # 尝试点击复制按钮
        # <i data-v-5240be39="" class="icon mtdicon mtdicon-copy-o"></i>
        log("正则未提取到，尝试点击 Copy 按钮...")
        copy_btns = driver.find_elements(By.CSS_SELECTOR, ".mtdicon-copy-o")
        if copy_btns:
            human_click(driver, copy_btns[0])
            time.sleep(0.5)
            try:
                # Mac pbpaste
                proc = subprocess.run(["pbpaste"], capture_output=True, text=True)
                clipboard_content = proc.stdout.strip()
                if clipboard_content.startswith("ak_") and '*' not in
clipboard_content:
                    key_to_save = clipboard_content
                    log(f"从剪贴板获取到 Key: {key_to_save}")
            except Exception as e:
                log(f"读取剪贴板失败: {e}")

    if key_to_save:
        with open(CONFIG["KEY_FILE"], "a") as f:
            f.write(key_to_save + "\n")
        log(">>> Key 已保存 <<<")
    else:
        log("未提取到有效 Key")
        driver.save_screenshot(f"extract_key_fail_{loop_index}.png")
        with open(f"debug_api_page_{loop_index}.html", "w") as f:
            f.write(driver.page_source)

    except Exception as e:
        log(f"提取 Key 异常: {e}")
        driver.save_screenshot(f"extract_error_{loop_index}.png")
log("提取 API Key...")
time.sleep(3)
try:
    # 刷新一下页面确保 Key 显示 (可选，有些SPA不需要)
    # driver.refresh()
    # time.sleep(2)

    # 获取页面原文找 ak_
    page_source = driver.page_source
    found_keys = re.findall(r'(ak_[a-zA-Z0-9_\-]+)', page_source)

```

```

        key_to_save = None
        if found_keys:
            # 过滤掉一些非 Key 的字符串 (key 长度通常较长)
            valid_keys = [k for k in found_keys if len(k) > 20]
            if valid_keys:
                # 假设最新的在最上面, 或者页面 source 顺序
                # LongCat 列表通常最新的在上面
                key_to_save = valid_keys[0]
                log(f"正则提取到 Key: {key_to_save}")

        # 尝试点击复制按钮
        # 鼠标悬浮到 Key 的行 (如果需要)
        if not key_to_save:
            rows = driver.find_elements(By.CSS_SELECTOR, ".el-table__row")
            if rows:
                # 悬浮第一行
                ActionChains(driver).move_to_element(rows[0]).perform()
                time.sleep(0.5)
                # 查找复制按钮
                copy_btn = rows[0].find_element(By.XPATH, ".*//*[contains(@class,
'copy') or contains(text(), '复制')]")
                human_click(driver, copy_btn)
                time.sleep(0.5)
                # 读取剪贴板
                proc = subprocess.run(["pbpaste"], capture_output=True, text=True)
                if proc.stdout.strip().startswith("ak_"):
                    key_to_save = proc.stdout.strip()
                    log(f"复制按钮提取到 Key: {key_to_save}")

        if key_to_save:
            with open(CONFIG["KEY_FILE"], "a") as f:
                f.write(key_to_save + "\n")
            log(">>> Key 已保存 <<<")
        else:
            log("未提取到有效 Key")
            driver.save_screenshot(f"extract_key_fail_{loop_index}.png")

    except Exception as e:
        log(f"提取 Key 异常: {e}")
        driver.save_screenshot(f"extract_error_{loop_index}.png")

    log("本轮结束, 准备下一轮 ... ")
    time.sleep(2)

except Exception as e:
    log(f"全局异常: {e}")
finally:
    if driver:
        driver.quit()
    log("浏览器已关闭")

if __name__ == "__main__":
    automation_loop()

```

在此处键入或粘贴代码